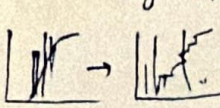
- Open CV : BGR ; Numpy Array (h,w,3) dtype: uint8 (Pixels) h×w×3×8
  - CCV.COLOR_BGR2LAB)
- Bayer Pattern (detects color) on photosites (intensity)
- 00 = 10×16+6 [FF = 15×16+15]
- RGB color space [Device Dependent
  - × Separate color from brightness.
  - × Perceptually Uniform → Equal distances in color space = equal diff. in our perception energy.
- Good → Device independent; map better to human perception; Intuitive Manipulation
- CHROMATICITY (A)
- H → Type of Color ; Saturation: Intensity, Value: Brightness (V. Axis)
- RGB → Negative red colors Thus x+z (x+y → chrom., Y → luminance) Color: (x,y,Y) (x+y+z=1)
  - x = X/X+Y+Z ; y = Y/X+Y+Z
- Lab → Perceptually Uniform

	constant	change
Scene brightness	HS ab	Y RGB
Diff colors	Y	HS RGB ab

- Make image more vibrant (A) → Use full dynamic range of display; Increase contrast using histogram equalization on luminance; Exaggerate colors by ↑ saturation



Histo-Equalization → tries to redistribute channel values so that each value in [0, 255] range is "equally" common.

- Cumulative sum (np.cumsum); CV.LUT (LOOKUP Table) (Y=X)
- Partial hist. eqn done by taking weighted avg. of identity LUT and equalization LUT (cumsum)

Perceptual similarity of 2 colors can be calculated by treating their L,a,b values as 3D coordinates and measuring Euclidean dist. b/w them

- Small aperture: less light gets through, diffraction effects.
- magnification $|m| = \frac{d_i}{d_o} = \frac{f}{z_o}$ [object → small → m: same; large → m changes throughout object body.]
- Persp. Proj^n → $p' = f\begin{bmatrix} x/z \\ y/z \end{bmatrix}$

Linear $p = \begin{bmatrix} x \\ y \end{bmatrix}$ Augmented $\bar{p} = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$ Homogeneous $\tilde{p} = \begin{bmatrix} wx \\ wy \\ w \end{bmatrix} = f\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}\begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$ linear function

- Sensor Coordinates < x: right, y: down | Camera Coord :→ +x & +y → orient^n of image sensor inside the camera.
- Change of basis: Expressing one point in another coordinate system.
  - → Matrix multiplication in homogeneous, augmented.
- $p' = [u' \; v']^T$ (P) (R) (p-O) ⇒ $p' = Rp + t$ (translation)
  - rot^n, shift

	Matrix	DOF
2D R: 2×2 ; t: 2×1	3×3	3 (2t+1R)
3D R: 3×3 ; t: 3×1	4×4	6 (3t+3R)

Homogeneous: $\tilde{p}' = \begin{bmatrix} R & t \\ 0^T & 1 \end{bmatrix}\tilde{p}$

Camera Calibration :→ (Stereo Vision/Panorama Stitching) $[R = [p_1 \; p_2 \; p_3]]$

- Camera Intrinsics (sensor to camera) (3×3 Calib^n Matrix)
- Camera Extrinsics (Camera to world) (3×4 pose relative to world coord.)
  - → pixel loc^n in sensor coord.
- Sensor → Camera.

Loc^n Transl^n Scaling

$p = [R_s \; t_s]\begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} x_s \\ y_s \\ 1 \end{bmatrix}$ Transformation Matrix

(6 DOF)

⇒ $p = M_s \bar{x}_s$ → 3D pts expressed in camera coord. relative to camera origin O_c

⇒ $p = \alpha p_c$

$\bar{x}_s = \alpha M_s^{-1} p_c$ = [K] p_c → [R t] p_w (6 dof)

3×3 calibration matrix. (as scale of $\bar{x}_s$ doesn't matter)

$\bar{x} = P\bar{p}_w$ (not invertible)

K → Info < Position, orientation of image plane relative to camera origin; Phy. size of pixels, conv. from real units to pixel units

→ DOF : scale in x & y. | If $s_x = s_y$; sensor xy parallel to camera xy → 4 DOF (3 position, 1 unit)

- Homography : Transformation matrix that defines a projective transformation b/w two images. (8 DOF → Homo. coord → ×scale)

- → preserves straight lines (× parallel, angles, sizes)
- least square regression – best fitting H.)
- warp Perspective : Warp image 1 as if viewed from perspective of image 2
- Lens eqn → $1/z_o + 1/z_i = 1/f$ ⇒ $z_i = f$ [ideal aperture] (+ve : Redn of Exposure Time)
- Non-linear distortions – Radial, tangential (camera intrinsics)

World → camera → intermediate → warped intermediate → sensor

linear (change of basis) linear (shift + scale)

- Camera Calibration [Homography → only rotating around origin, not translating, flat plane)

Preprocessing → Calibration × removing non-linear distortions; cropping – resizing ; Noise Redn ; color correction – contrast enhancement, feature extraction)

- Filtering < Point Based : (single pxl at a time) → Histo. Eqn (constant), Brightness.
  - 2D Filtering → modify each pixel as function of its surroundings.
- Box filter (correlation) → 5×5 filter but its acts as 3×3 filter. (just window size)
  - → Pros : Fast (only averaging) ; parameter light ; effective for AWGN Redn
  - → Cons : o/p image has box effect ; blur edges, treats all pts in window as same ; outliers / sudden changes strongly affect result. (sum inside should be 1)
- Gaussian Filter < neighbouring pixels've most influence (↑ we). ; σ gives direct control over which blur we apply
  - ↑σ → wider kernel → more smoothing
  - ↑5σ : background esp^n.
  - ↑ edges vanish

Thumb Rule. ∘ Edge^s values ≈ 0 ∘ Filter half-width = 3×σ

no neighbour pixels

(clip, wrap around, copy, reflect)

- conv. shifted left by 1 px ; (corr) → shifted right by 1 px

flip filter left to right & top to bottom (doesn't matter → kernel: symmetric)

- Commutative, associative, distributive, identity $h[k,l]$ / (useful when cascading multiple conv. or. corr. filters] $= h[-k,-l]$
  - (corr. & conv. – both can be scaled) → separable → outer product of vector u & v.
- correlation < $m^2 n^2$ Multiplications → mn ; $m^2 - 1$ Additions → $2(m-1)$
- Bilateral Filtering → preserves edges (by not averaging across edge)
  - ↳ upto now → considered only spatial closeness. (Domain Kernel)
  - ↳ Range Kernel : → intensity similarity → when constant (luminance) behaves like Gaussian blur
  - → Inseparable (loop over i,j,k,l)
  - → Does blurring → spreads out s&p noise.
- Salt & Pepper Noise → ✓ Median filter.
- Sharpening : opposite of blurring. ($g_{blur} = f * h$)

$g_{sharp} = f + \alpha(f - (f * h))$ = $f * h_{sharp}$ (single filter) $h_{sharp} = \delta(1+\alpha) - \alpha h$

Template Matching → correlation to score each loc^n (similarity) → ① cv.matchTemplate() → cv.minMaxLoc() (& loc^n of best matching pixel) & (1+α) − αh

(reject if score < threshold)

② → Thresholding → NMS → matches should only suppress each other if they are similar → Reject all loc^n with better scoring neighbour

m^2 → if we ↑ template size → ↑ cost ↑ with temp. area (m^2)

→ cost ↓ " image area (M^2)

IOU = Area of Intersection / Area (Union)

→ Multiscale image pyramid to achieve translation & scale invariance

③ multi-scale pyramid rep^n → Use same (Use gaussian blur to remove noise before template matching)

sub-sampling: high freq pts.

• Precision : TP/TP+FP | Recall : TP/TP+FN | F1 = 2PR/P+R

• operations on edges are efficient [Discontinuities → depth : bg/fg, surface color, illumination : shadows, surface orient^n : phy edges]
[differentiating a discrete image using finite differences. derivatives are large here. filter : [-1, 0, 1]]

• 1st derivative : Noisy → use Gaussian filter (convolve & then der → clear peak) ① convolution ② Conv
• Differentiable convolution (Better) → apply conv. to de-noised, edge detection filter → $\frac{d}{dx}(f*g) = f * \frac{dg}{dx}$ (only one conv.)

• Sobel = Horizontal Blurring + 1D derivative filter → changes along x-axis → vertical edges

• vertical sobel : [1 0 -1] * [1 2 1] → horizontal edges.

Roberts cross $\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$ and $\begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$

• Pre-wrt : Replace 2 with 1.

• HSV ① Edge direction → Edge mag.
• for a given pixel → ① Run both horizontal & vertical edge detectors ② Normalize it mag (mag. max()) ③ HSV

• Canny Edge Detector
① convert to single channel image ② Filter image (Gaussian Blurring) ③ Gradient Calculation (sobel)
④ Conversion to mag & direction ⑤ NMS (direction dependent - only higher edges (make edges thinner) ⑥ Double Thresholding - Use high threshold to start edge curves & low threshold to continue them ⑦ Hysteresis : Promoting weak edges to strong edges if they are connected to strong edges (Rank edges & connect strong & weak edges × Noise)

[Grad mag > high thresh : Strong edge ① edge → second der = 0 Laplacian
" " < low " : Noise
In between : Weak.

Boundary detection (detect edges & reason about them) → Issues - Noise, clutter, incomplete edge/boundary, extra edge pts, direction.
1st der : if there are edges
II der : Compute exact location of edges

① Strawman Sol^n (Least square line fitting) ↳ minimizes vertical distance b/w all edge pts → ② to use

② Minimize ⊥ dist. ($x\cos\theta + y\cos\theta = \rho$) [1]

Outliers → Robust Regression, RANSAC (Random Sample Consensus)
→ Subset of points, fit a line - ask how many pts are close to that line (consensus) repeat N times, keep cons > threshold.

→ Hough Transform
Take point in image space & convert to line in parameter space. (all lines in parameter space : Intersect → line in image space) (works even in presence of occlusions)

Line detection Algo
Quantize parameter space → create accum. array $CA(m,c) = 0 \forall m,c$ → Each edge pt → find local maxima in $A(m,c)$
$A(m,c) = A(m,c) + 1$ if (m,c) lies on l ($c = mx_i + y_i$)

But m - infinite [breaks for vertical line, large accumulator, more memory & computation] → SINUSOIDS. ρ, θ = finite.

Acc
↳ Too big : diff^t lines merged together → can create false peaks making it difficult to distinguish real lines.
↳ Too small : Noise can cause lines to be missed

• Why simple shapes only?
① No. of parameters of shape = dimensionality of accumulator array.
② Domain knowledge
③ Use edge direction info - possible to do any radius circle detection with 2D accum.

• for circles ① If we edge gradient, we know circles are going to lie on this line → reduces search space.
② Used for NMS (direction - dependent) : Canny Algo.

• Binary image : Benefits → Reduced storage space, faster processing, simplified analysis, noise red^n, cost efficiency.
• cv.threshold (img, thresh_value, max_value, thresh_type) max_value = 255, otherwise.
• Adaptive Thresholding → useful when lighting varies across images.
• Moments → geometric properties of shape
↳ (0,0) moment → Area.
↳ 1st Order ⇒ $\bar{x} = \frac{M_{10}}{M_{00}}$; $\bar{y} = \frac{M_{01}}{M_{00}}$ → centre of mass
↳ II order ⇒ orientation & Roundness. (Short Axis / Long Axis)

• Connected Component Labeling → (a) find unlabeled seed pt, b=1. If not found, terminate (b) Assign new label to seed pt. (c) Assign same label to its neighbours b=1 (c) N. of neigh b=1 (d) Repeat until no more unlabeled neighbours with b=1 (e) Go to (a)
• 4C ignores thin lines of 1s joining 2 objects.
• 8C ignores thin lines of 0s separating 2 objects.
• 6C : Neighbours on hexagonal grid. (All are face neighbours & not corner neighbours (squash hexagonal grid back to rectangular grid.)
• detect dark ink on white bg → cv.cvtColor(image, cv.COLOR_BGR2GRAY) < 142
• Opening : dilate(erode(image, s), s), Closing : erode(dilate(...))
• sequential labeling Algo : - Raster scanning
→ morphological operations → Mathematical operations applied to binary/grayscale images to refine shape of object & remove noise.
→ dilation : Expands connected sets of 1s → Growing features, filling holes & gaps
→ Erosion : shrinks features → removing bridges, branches, protrusions
• Structuring element : Any shape/size digitally representable.

Dilate: B ⊕ S

Erode: B ⊖ S

Object Tracking Background Subtraction Algo → Build bg image by taking avg of K frames. (subtract bg from every frame - Thresholding)
→ take median to build bg image from K frames. Then (Not much noise but intensity value can change substantially across multiple frames)
→ Adaptive median → Take median from last K frames → expensive because of per pixel opns every frame (Better)
→ Pixel Intensity distribution (GMMs) → using distribution → classify px → bg / fg.

ω → scale ; σ → width
[R,G,B] → mu - vector ; σ → co-variance matrix

GMModel = fitgmdist(K, K) :→ GM dist. model with K components fitted to data X.
→ Large K : unstable
→ Small K : X flexible
① Offline : compute GMM for all pixels - classify each Gaussian (bg / fg)
Online → 1st X image → OFF
↳ find nearest Gaussian to pixel x (dist < 2.5σ) → classify that pixel using gaussian label. Update hist. with new x (don't re-compute GMM - only replace histogram & compute GMM only if change is large)

• F(u) holds amplitude & phase of sinusoid of freq u. (cpmplex value in numpy)

• cos → apne, sin → apne/reche, i → i(pi) $f(x) \to 1$, $e^{-ix} \to i$

	Spatial Domain	Freq domain
Linearity	$\alpha f_1(x) + \beta f_2(x)$	$\alpha F_1(u) + \beta F_2(u)$
Scaling	$f(ax)$	$\frac{1}{\lvert a \rvert} F\left(\frac{u}{a}\right)$
Shifting	$f(x-a)$	$e^{-i2\pi ua} F(u)$
Differentiation	$\frac{d^n f(x)}{dx^n}$	$(i2\pi u)^n F(u)$

Separable $O(N^2M)$
conv. $(O(N^2 \log N))$
ansep. $(O(N^2M))$
2D → 1D conv → $O(NM)$
FFT: $O(M \log M)$
FFT: $O(HW \log H + \log W)$

• $G(u) = F(u)H(u)$ ($g(x) = f(x) * h(x)$) (vice versa)
IFT, FT

If zones are far apart, dots are closer.
[Using conv. theorem - regardless (H) → sep^n]
→ also, no padding reqd.

• Natural images → scale invariant freq spectrum [Power spectrum $= \lvert F(u,v) \rvert^2$]
↳ $1/f^2$ power spectrum (coiner^n bias)
→ Spectral density of signal: Power as function of frequency. discards phase info.

• Aliasing - Undersampling: Signal sampled at a rate too low to capture its high freq, misrep. as low freq → artifacts
Gaussian blur.
→ ① Oversample ② Remove high freq. before sampling.
(creating Gaussian pyramid → blurring before downsampling)

• Signal → max freq → f_{max} (min pixel freq) → largest sampling spacing w/o aliasing
$f_{sample} = 2 f_{max}$
↳ occurs when imaging a scene that has frequen^T image sensor's Nyquist freq (0.5 cycles) ($1/2$ → sampling rate)

• Deconvolution - can amplify weakened high freq. (can recover much of the detail) $F'(u,v) = \frac{G(u,v)}{H(u,v)}$ → IFT → $f'(x,y)$

Issues → if $H(u,v) = 0$, $F'(u,v) = \infty$ - Not Recoverable.
→ when we divide by very small H(u,v) → we get huge amplification → Noise gets amplified by $1/H(u,v)$ → Noise Suppression: Wiener Deconvolution

Noise to Signal Ratio (NSR) $= \boxed{\gamma = 0.002}$ - $\frac{\text{Power of noise at } (u,v)}{\text{" signal " "}} = \frac{\lvert N(u,v) \rvert^2}{\lvert F(u,v) \rvert^2}$

Corner (well defined x & y, size ✗); Blob → x, y, size ✓

• Harris corner detection → compute descriptors of image R → Edge detection using Laplacian (Compute G_x & G_y → compute $\Sigma[i,j]$ local variation in gradients in window around each focus → compute λ_1, λ_2 singular values of Σ, compute $R = \lambda_1\lambda_2 - k(\lambda_1 + \lambda_2)$, Threshold R, NMS)
↳ 2nd order der. of Gaussian signal → conv. (I & Laplacian) → zero crossing (edge/centre of blob) loc^n of blob
gives precise location of edge.

• Changing normalization factor → fmp (response) ↑ until peak
→ for given σ, if we've peak → loc^n of blob.
scale parameter (scale) ✓

• Convolve image with NLOG operator at diff^t scales → from stack of images find local extrema in the stack., σ: scale of blob.
Position of blob.

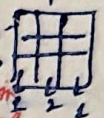
• Laplacian of Gaussian → Expensive ∴ → DOG (Diff.) → find extremum in every 3x3 grid → Interest Point (Thresholding & Stability)
and Data → SIFT Interest Pts.

• SIFT Descriptor: Apply gradient operator to all px within keypt → divide area into 4 quadrants → create histogram for orientations at each quadrant. (Orient^n with max value = ori. of keypt)
↓
Invariant to brightness, scale & rot^n. ↑ Entire disc. window rotated so that principal ori = 0°
→ Hist. dist. $H_1(k)$ & 1st dis. $H_2(k)$ → Σ 0 Euc. dist → perfect match.

• RANSAC (sample subset of datapts → compute model parameters [inliers] (Check rch. pts & fit a line & check remaining pts that are inliers) → score the model based on no. of inliers → select model with highest score & repeat
→ Line = 2; circle: 3, Homographies: 8

→ Image → Gaussian filter → Low Pass.
↳ Laplacian → High Pass
contains band pass freq ($f_k = f_k - f_{k+1}$)

• Laplacian Pyramid → compute difference b/w upsampled Gaussian pyramid level (k+1) & k. (Upsample)
(Place zeroes in b/w)



Encoder/Analysis
$G_0 \to G_1 \to G_2$, $F_2 \to F_1 \to F_0$
L_0, L_1, L_2
Decoder/Synthesis

→ Image compression
→ Noise Removal
→ Computing image features
→ Image blending
① Create Gau. Pyr.
② From each GP create LP.
③ At each level k:
$M_k L_{A,k} + (1 - M_k) L_{B,k}$ = $L_{blend,k}$
↳ mask at that scale.
↳ smoothened at coarser levels
④ Collapse blended LP by successive upsampling & adding.

• Each px value = dist. from optical origin of camera [image (h,w,3) / depth map (h,w)]

• Stereo vision: Take 2 views of scene from diff^t perspectives → 3D loc^n: where rays intersect (Triangulation) - Using time of flight measures
• LiDAR, $d = c * t/2$

• Correspondence Problem → Panorama stitching, Obj. rec/tracking, Depth, Motion est^n, segmen.
↓
finding matching pts from 2D search to 1D search
[Rectification → all epipolar lines - horizontal]
↓
Project images onto common plane

• Disparity: distance b/w pixel locations of pt in both cameras.
$Z = f * T / D * d$ [Disparity ↑↑ → objects closer]

• Essential matrix and Epipolar constraint eqn.
$E = [t] \times R'$ ⇒ $\hat{x}_1^T E \hat{x}_0 = 0$ → can't recover [fully determined]
↳ Info about camera pose (R,t); expresses epipolar constraint in both directions; given many corresponding x_0 & x_1 → solve for E → requires RANSAC if only probable correspondences are known
↳ We can recover R and unit vector pointing in t's direction

• Distance to Ray Method (Triangulation in OpenCV) → DLT (Direct Linear Transform)
→ Triangulates 3D position of 2d correspondences b/w several images
→ Pros: fast; Analogous to solving for homography
→ Cons: All errors from all cameras treated equally regardless of dist. to object.

- Motion field → vector field on the image plane where each 2D xy point has corresponding 2D key velocity → object moves but pixel doesn't change
- Optical field → 2D vector field representing motion of objects in the image. brightness patterns
  - Ideal → Equal
  - Practical → differ

Pixels change but object doesn't move

- Optical Mouse
- Traffic Monitoring
- Video Stabilization

- Aperture Problem – we cannot locally recover true motion field in regions of image that are edge-like
- Stereo → epipolar constraint, 1 parameter: disparity
- OF → objects can move in any direction: 2 par (V_x, V_y)
  - Assumptions
    - Brightness Constancy Constraint $I(x,y,t) = I(x+u, y+v, t+1)$
    - Small motion $|u|, |v| \to$ small
      - Taylor Series $\frac{dI}{dx}$, $\frac{dI}{dy}$ (image gradients by Sobel)
- Optical flow constraint eqn ⇒ $I_x u + I_y v + I_t = 0$
  - Lucas Kanade sol^n → Spatial Coherence Constraint → Neighbouring pixels have same (u,v) → This gives us overdetermined system that can be solved using least squares
  - → Component of flow ⊥ to gradient – can't be recovered.

$(A^T A)d = A^T d$ → invertible, well-conditions

- Moment Matrix ~ Harris corners. Discard bad pts & add in new good pts.

Sparse Motion → Use good features → Lucas Kanade → Track pts over multiple frames

Multiscale Optical Flow Algo → Create image pyramid until enough levels that motion is small at low resolution → calculate optical flow for lowest res level of pyr. using LK → Upsample the flow (with scaling) → Calculate warped frame [k] based on flow from previous level → Calculate from from w-f [k] to frame [k+1] → This is residual motion not explained by the previous level → Update flow (Upsampled flow from prev. level + residual motion this level) → Go to ~ → Catch errors. Set flow to 'nan' or 0.

Inductive Bias → domain know)
- Locality, Trans-Invariance, Hierarchy)

Flow → Corresp. are good?

(a) Lowe Ratio Test: for every best match, check second best match. If ratio ≈ 1 (drop)

(b) Cross-Check: find best scoring match for left feature in the right image. Check that feature from left image is also best match for right. Drop anywhere they are not each other's best matches

- world stationary, camera moving → just 1 param. to estimate per pixel (depth) + global "speed" Param = P+1 unknowns / frame
- Objects undergoing planar motion → express (u,v) as homography and estimate 8 parameters for homo matrix for all px on that plane
- Camera rot^n (world st^{ty})

- 2 Optimization Prob → Triangulation, Variational Methods, Texture Synthesis
- → Optimising a single O/P to meet some criterion/Obj.
- → learning a model that works well for many diff I/Os

- Gradient Descent O/P | Jacobian → both vectors
- Hessian → both scalar
- Numpy/Array (Shape, dtype) Tensor (device, requires_grad)
- Quadratic cost → often nice analytically but sensitive to outliers.
- Regression – fit a model to some data by minimizing some cost/energy function.
- x.backward() populates x.grad & keeps graph around (retain_graph = True)
- SAD approach to disparity → encourage match in appearance (Photometric cost) (Data dependent energy term)
  - ↳ Encourage disparity of nearby pixels to be similar (Smoothness cost)

→ Variational disparity: simultaneously optimize correspondence & smoothness

Choose hyperparameters → Get initial estimate of disparity → Calculate data dependent error term + Reg. Term → Calculate error gradient & update → check for conv. or go to 3

- Differentiable model → compute gradient of loss wrt pixels.
- Semantic Seg → what type of thing is where; Instance → count/track individual
- → Panoptic: Identify separate obj & classify them

Energy fun: $E = E_{data} + \lambda E_{smoothness}$
- E_{data} → how well each pixel fits cluster
- $E_{smoothness}$ → Consistency b/w neighbouring pixels; Penalty for cutting b/w similar neigh. pixels.
- λ → balance scale

(K-Means), graph cuts (similar appearance → ↑ wt → find minimum cut that separates graph into groups while cutting least total edge wt).

Energy = Σ wt of remaining links to T×S + Σ wt cut neighbour links

$E = E_{data} + E_{boundary}$ → Penalizes strong connections b/w similar pixels.

[Variational method would encourage smoothness in disparity map by including reg. term whereas with SAD – smooth results – large window size]

- Template M → Doesn't see what digit is it ← sum → ✗ Non-separable value
- Augment → torch.transforms (PD-const.
- Classif. → loss func: cross entropy
- Conv → feature maps → sub fmaps
- PyTorch → reverse mode automatic differentiation. (Auto Grad)

Rules
- Start simple – create a benchmark [NN.linear(in_size, out_size)]
- Manually review i/ps & o/ps.
- You never train one model.
- Checkpt & log.
- Statistical honesty.

h×w. (x.view-we done)

- ReLU – works better when layers are wide. assign no any memory
  - ↳ last layer → Accuracy ↓ (forces all o/ps ≥ 0) unlike np.reshape()
- NN's → ↑ expressive → ↑ D.
- ↳ conv → x.pad(k, pad) (c corr.)
  - $\sum_{k,l,c} im[c, l+k, j+l]\, w[j,c,k,l]$ = out[j,i,j]
  - channels → RGB.
  - filter index.
- → translation invariance.
- Fully Connected Neuron → Many Parameters, ✗ Locality, ✗ Trans. Invari, No. of filters
  - (32,32,3) → (b,c4,32,32) Ignores → nearby pixels – relate.
- Weight Sharing – Use same n/w parameters to detect local patterns at many locations in image. (Hierarchy – local low level features are composed into larger more abstract features).
- → Locally connected → 1 parameter per Each local patch gets diff^t set of weights – so n/w learns separate detectors for every loc^n → ✗ generalisat^n
- Trainable Parameters = (In Channels × out Kernel H × K W + Bias) Out-Channels. (Receptive field = kernel)
- VGG → allows more non-linearities → deeper.